CS23431 - OPERATING SYSTEM

EXP 10: Page Replacement Algorithms

QUESTION :

Write a C program to implement the **FIFO Page Replacement Algorithm**. The program should take the number of pages, reference string, and number of frames as input. Simulate the page replacement process by replacing the page that entered memory first whenever a page fault occurs and no empty frame is available. Finally, display the frame status after each page reference and calculate the total number of page faults.

CODE:

```
#include <stdio.h>

int main() {
    int pages[50], frames[10], n, f, i, j, k = 0, fault = 0, found;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter reference string: ");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++)
        frames[i] = -1;

    for(i = 0; i < n; i++) {
        found = 0;

        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        if(!found) {
            frames[k] = pages[i];
            k = (k + 1) % f;
```

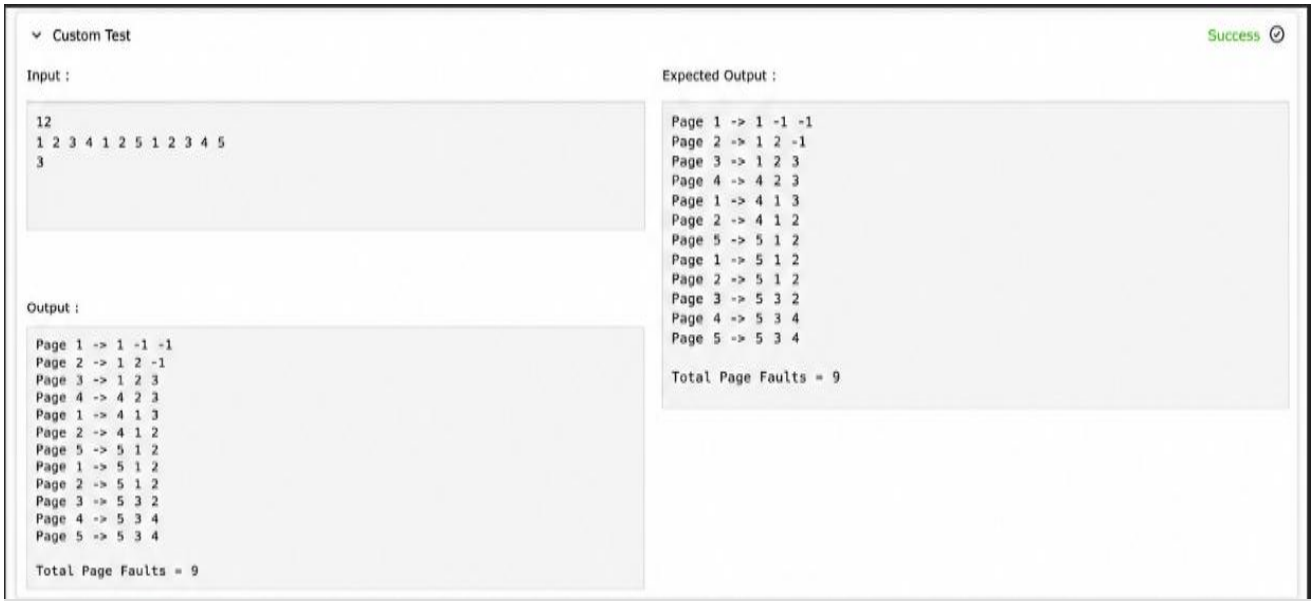
```
            fault++;
        }

        printf("Page %d -> ", pages[i]);
        for(j = 0; j < f; j++)
            printf("%d ", frames[j]);
        printf("\n");
    }

    printf("Total Page Faults = %d\n", fault);

    return 0;
}
```

OUTPUT :



QUESTION :

Write a C program to implement the **LRU (Least Recently Used) Page Replacement Algorithm**. The program should take the number of pages, reference string, and number of frames as input. For each page reference, replace the page that has not been used for the longest period whenever a page fault occurs and all frames are occupied. Finally, display the frame status after each page reference and calculate the total number of page faults.

CODE:

```
#include <stdio.h>

int main() {
    int pages[50], frames[10], time[10];
    int n, f, i, j, pos, fault = 0, found, least;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter reference string: ");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++) {
        frames[i] = -1;
        time[i] = 0;
    }

    for(i = 0; i < n; i++) {
        found = 0;

        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                found = 1;
                time[j] = i;
                break;
            }
        }

        if(!found) {
            least = time[0];
            pos = 0;

            for(j = 1; j < f; j++) {
                if(time[j] < least) {
                    least = time[j];
                    pos = j;
                }
            }

            frames[pos] = pages[i];
            time[pos] = i;
            fault++;
        }

        printf("Page %d -> ", pages[i]);
        for(j = 0; j < f; j++)
            printf("%d ", frames[j]);
        printf("\n");
    }

    printf("Total Page Faults = %d\n", fault);
```

```
    return 0;
}
```

CODE :

QUESTION :

Write a C program to implement the **Optimal Page Replacement Algorithm**. The program should take the number of pages, reference string, and number of frames as input. For each page reference, replace the page that will not be used for the longest period in the future whenever a page fault occurs and all frames are occupied. Finally, display the frame status after each page reference and calculate the total number of page fault.

CODE:

```
#include <stdio.h>

int main() {
    int pages[50], frames[10];
    int n, f, i, j, k, pos, fault = 0, found, farthest;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter reference string: ");
    for(i = 0; i < n; i++)
```

```
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++)
        frames[i] = -1;

    for(i = 0; i < n; i++) {
        found = 0;

        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        if(!found) {
            farthest = -1;
            pos = -1;

            for(j = 0; j < f; j++) {
                for(k = i + 1; k < n; k++) {
                    if(frames[j] == pages[k]) {
                        if(k > farthest) {
                            farthest = k;
                            pos = j;
                        }
                        break;
                    }
                }

                if(k == n) {
                    pos = j;
                    break;
                }
            }

            if(pos == -1)
                pos = 0;

            frames[pos] = pages[i];
            fault++;
        }

        printf("Page %d -> ", pages[i]);
        for(j = 0; j < f; j++)
            printf("%d ", frames[j]);
        printf("\n");
    }

    printf("Total Page Faults = %d\n", fault);

    return 0;
}
```

OUTPUT:

Custom Test

Success

Input :

```
9
4 1 2 5 1 2 3 5 4
3
```

Output :

```
Page 4 -> 4 -1 -1
Page 1 -> 4 1 -1
Page 2 -> 4 1 2
Page 5 -> 4 1 5
Page 1 -> 4 1 5
Page 2 -> 4 2 5
Page 3 -> 3 2 5
Page 5 -> 3 2 5
Page 4 -> 4 2 5

Total Page Faults = 7
```

Expected Output :

```
Page 4 -> 4 -1 -1
Page 1 -> 4 1 -1
Page 2 -> 4 1 2
Page 5 -> 4 1 5
Page 1 -> 4 1 5
Page 2 -> 4 2 5
Page 3 -> 3 2 5
Page 5 -> 3 2 5
Page 4 -> 4 2 5

Total Page Faults = 7
```